```
import pandas as pd
import numpy as np

from google.colab import files

uploaded = files.upload()
```

```
<IPython.core.display.HTML object>
```

```
Saving WELFake_Clean.csv to WELFake_Clean.csv
```

```
df = pd.read_csv("WELFake_Clean.csv")

print("Dataset Loaded Successfully!\n")

print("Shape:", df.shape)

print("\nColumns:")
print(df.columns)

print("\nFirst 5 Rows:")
display(df.head())
```

```
Dataset Loaded Successfully!

Shape: (71537, 3)

Columns:
Index(['title', 'text', 'label'], dtype='object')

First 5 Rows:
```


```
{"summary":"{\n  \"name\": \"display(df\",\n  \"rows\": 5,\n
\"fields\": [\n    {\n      \"column\": \"title\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 5,\n        \"samples\": [\n
\"UNBELIEVABLE! OBAMA\\u2019S ATTORNEY GENERAL SAYS MOST CHARLOTTE
RIOTERS WERE \\u201cPEACEFUL\\u201d PROTESTERS\\u2026In Her Home State
Of North Carolina [VIDEO]\",\n          \"About Time! Christian Group
Sues Amazon and SPLC for Designation as Hate Group\",\n
\"Bobby Jindal, raised Hindu, uses story of Christian conversion to
woo evangelicals for potential 2016 bid\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"text\",\n      \"properties\": {\n
\"dtype\": \"string\",\n        \"num_unique_values\": 5,\n
\"samples\": [\n          \" Now, most of the demonstrators gathered
last night were exercising their constitutional and protected right to
peaceful protest in order to raise issues and create change.
Loretta Lynch aka Eric Holder in a skirt\",\n          \"All we can
say on this one is it s about time someone sued the Southern Poverty
Law Center!On Tuesday, D. James Kennedy Ministries (DJKM) filed a
```

lawsuit against the Southern Poverty Law Center (SPLC), the charity navigation organization GuideStar, and Amazon, for defamation, religious discrimination, and trafficking in falsehood. The SPLC listed DJKM as a hate group, while GuideStar also categorized it in those terms, and Amazon kept the ministry off of its charity donation program, Amazon Smile. We embarked today on a journey to right a terrible wrong, Dr. Frank Wright, president and CEO at DJKM, said in a statement Tuesday. Those who knowingly label Christian ministries as hate groups, solely for subscribing to the historic Christian faith, are either woefully uninformed or willfully deceitful. In the case of the Southern Poverty Law Center, our lawsuit alleges the latter. The SPLC has labeled DJKM an anti-LGBT hate group for its opposition to same-sex marriage and transgenderism. These false and illegal characterizations have a chilling effect on the free exercise of religion and on religious free speech for all people of faith, Wright declared. After having given the SPLC an opportunity to retract, we have undertaken this legal action, seeking a trial by a jury of our peers, to preserve our own rights under the law and to defend the religious free speech rights of all Americans, the DJKM president concluded.The lawsuit laid out charges against the SPLC, GuideStar, and Amazon.Read more: PJM\",\n   \"A dozen politically active pastors came here for a private dinner Friday night to hear a conversion story unique in the context of presidential politics: how Louisiana Gov. Bobby Jindal traveled from Hinduism to Protestant Christianity and, ultimately, became what he calls an \\u201cevangelical Catholic.\\u201d\\n\\nOver two hours, Jindal, 42, recalled talking with a girl in high school who wanted to \\u201csave my soul,\\u201d reading the Bible in a closet so his parents would not see him and feeling a stir while watching a movie during his senior year that depicted Jesus on the cross.\\n\\n\\u201cI was struck, and struck hard,\\u201d Jindal told the pastors. \\u201cThis was the Son of God, and He had died for our sins.\\u201d\\n\\nJindal\\u2019s session with the Christian clergy, who lead congregations in the early presidential battleground states of Iowa and South Carolina, was part of a behind-the-scenes effort by the Louisiana governor to find a political base that could help propel him into the top tier of Republican candidates seeking to run for the White House in 2016.\\n\\nKnown in GOP circles mostly for his mastery of policy issues such as health care, Jindal, a Rhodes Scholar and graduate of the Ivy League\\u2019s Brown University, does not have an obvious pool of activist supporters to help drive excitement outside his home state. So he is harnessing his religious experience in a way that has begun to appeal to parts of the GOP\\u2019s influential core of religious conservatives, many of whom have yet to find a favorite among the Republicans eyeing the presidential race.\\n\\nOther potential 2016 GOP candidates are wooing the evangelical base, including Sens. Rand Paul (Ky.) and Ted Cruz (Tex.) and Indiana Gov. Mike Pence.\\n\\nBut over the weekend in Lynchburg \\u2014 a mecca of sorts for evangelicals as the home of Liberty University, founded in the 1970s

by the Rev. Jerry Falwell \\u2014 Jindal appeared to make progress.\\n\\nIn addition to his dinner with the pastors, he delivered a well-received \\u201ccall to action\\u201d address to 40,000 Christian conservatives gathered for Liberty\\u2019s commencement ceremony, talking again about his faith while assailing what he said was President Obama\\u2019s record of attacking religious liberty.\\n\\nThe pastors who came to meet Jindal said his intimate descriptions of his experiences stood out.\\n\\n\\u201cHe has the convictions, and he has what it takes to communicate them,\\u201d said Brad Sherman of Solid Rock Christian Church in Coralville, Iowa. Sherman helped former Arkansas governor Mike Huckabee in his winning 2008 campaign for delegates in Iowa.\\n\\nAnother Huckabee admirer, the Rev. C. Mitchell Brooks of Second Baptist Church in Belton, S.C., said Jindal\\u2019s commitment to Christian values and his compelling story put him \\u201con a par\\u201d with Huckabee, who was a Baptist preacher before entering politics.\\n\\nThe visiting pastors flew to Lynchburg over the weekend at the invitation of the American Renewal Project, a well-funded nonprofit group that encourages evangelical Christians to engage in the civic arena with voter guides, get-out-the-vote drives and programs to train pastors in grass-roots activism. The group\\u2019s founder, David Lane, has built a pastor network in politically important states such as Iowa, Missouri, Ohio and South Carolina and has led trips to Israel with Paul and others seeking to make inroads with evangelical activists.\\n\\nThe group that Lane invited to\\u00a0Lynchburg included Donald Wild\\u00admon, a retired minister and founder of the American Family Association, a prominent evangelical activist group that has influence through its network of more than 140 Christian radio stations.\\n\\nMost of the pastors that Lane\\u2019s organization brought to Lynchburg had not met Jindal. But they said he captured their interest recently when he stepped forward to defend Phil Robertson, patriarch of the \\u201cDuck Dynasty\\u201d television-show family, amid a controversy over disparaging remarks he made about gays in an interview with GQ magazine.\\n\\nThroughout his Lynchburg visit, Jindal presented himself as a willing culture warrior.\\n\\nDuring his commencement address Saturday, he took up the cause of twin brothers whose HGTV reality series about renovating and reselling houses, \\u201cFlip It Forward,\\u201d was canceled last week after a Web site revealed that they had protested against same-sex marriage at the 2012 Democratic National Convention in Charlotte.\\n\\nThe siblings, Jason and David Benham, both Liberty graduates, attended the graduation and a private lunch with Jindal, who called the action against them \\u201canother demonstration of intolerance from the entertainment industry.\\u201d\\n\\n\\u201cIf these guys had protested at the Republican Party convention, instead of canceling their show, HGTV would probably have given them a raise,\\u201d Jindal said as the Liberty crowd applauded.\\n\\nHe cited the Hobby Lobby craft store chain, which faced a legal challenge after refusing to provide employees with insurance coverage for contraceptives as required under the Affordable Care Act. Members of

the family that owns Hobby Lobby, who have become heroes to many religious conservatives, have said that they are morally opposed to the use of certain types of birth control and that they considered the requirement a violation of their First Amendment right to religious freedom.\\n\\nThe family was \\u201ccommitted to honor the Lord by being generous employers, paying well above minimum wage and increasing salaries four years in a row even in the midst of the enduring recession,\\u201d Jindal told the Liberty graduates. \\u201cNone of this matters to the Obama administration.\\u201d\\n\\nBut for the pastors who came to see Jindal in action, the governor\\u2019s own story was the highlight of the weekend. And in many ways, he was unlike any other aspiring president these activists had met.\\n\\nPiyush Jindal was born in 1971, four months after his parents arrived in Baton Rouge, La., from their native India. He changed his name to Bobby as a young boy, adopting the name of a character on a favorite television show, \\u201cThe Brady Bunch.\\u201d\\n\\nHis decision to become a Christian, he told the pastors, did not come in one moment of lightning epiphany. Instead, he said, it happened in phases, growing from small seeds planted over time.\\n\\nJindal recalled that his closest friend from grade school gave him a Bible with his name emblazoned in gold on the cover as a Christmas present. It struck him initially as an unimpressive gift, Jindal told the pastors.\\n\\n\\u201cWho in the world would spend good money for a Bible when everyone knows you can get one free in any hotel?\\u201d he recalled thinking at the time. \\u201cAnd the gold lettering meant I couldn\\u2019t give it away or return it.\\u201d\\n\\nHis religious education reached a higher plane during his junior year in high school, he told his dinner audience. He wanted to ask a pretty girl on a date during a hallway conversation, and she started talking about her faith in God and her opposition to abortion. The girl invited him to visit her church.\\n\\nJindal said he was skeptical and set out to \\u201cinvestigate all these fanciful claims\\u201d made by the girl and other friends. He started reading the Bible in his closet at home. \\u201cI was unsure how my parents would react,\\u201d he said.\\n\\nAfter the stirring moment when he saw Christ depicted on the cross during the religious movie, the Bible and his very existence suddenly seemed clearer to him, Jindal told the pastors.\\n\\nJindal did not dwell on his subsequent conversion to Catholicism just a few years later in college, where he said he immersed himself in the traditions of the church.\\n\\nHe touched on it briefly during the commencement address, noting in passing that \\u201cI am best described as an evangelical Catholic.\\u201d Mostly, he sought to showcase the ways in which he shares values with other Christian conservatives.\\n\\n\\u201cI read the words of Jesus Christ, and I realized that they were true,\\u201d Jindal told the graduates Saturday, offering a less detailed accounting of his conversion than he had done the night before with the pastors. \\u201cI used to think that I had found God, but I believe it is more accurate to say that He found me.\\u201d\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n

```
}\n    },\n    {\n      \"column\": \"label\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\": 0,\n
\"min\": 0,\n        \"max\": 1,\n        \"num_unique_values\": 2,\n
\"samples\": [\n          0,\n          1\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe"}
```

```
print("Dataset Information")
print("="*50)

print(df.info())

print("\nMissing Values")
print(df.isnull().sum())

print("\nLabel Distribution")
print(df["label"].value_counts())
```

```
Dataset Information
==================================================

---------------------------------------------------------------------------
-----
NameError                                 Traceback (most recent call
last)
/tmp/ipykernel_1352/1326368457.py in <cell line: 0>()
      2 print("="*50)
      3
----> 4 print(df.info())
      5
      6 print("\nMissing Values")

NameError: name 'df' is not defined
```

```
!pip install nltk
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.13/dist-
packages (3.9.1)
Requirement already satisfied: click in
/usr/local/lib/python3.13/dist-packages (from nltk) (8.5.0)
Requirement already satisfied: joblib in
/usr/local/lib/python3.13/dist-packages (from nltk) (1.6.0)
Requirement already satisfied: regex>=2021.8.3 in
/usr/local/lib/python3.13/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.13/dist-
packages (from nltk) (4.67.3)
Requirement already satisfied: cloudpickle>=3.0 in
/usr/local/lib/python3.13/dist-packages (from joblib->nltk) (3.1.2)
```

```
import re
import string
```

```
import nltk

from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
```

```
True
```

```
df = pd.read_csv("WELFake_Clean.csv")

print(df.shape)

df.head()
```

```
(71537, 3)
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 71537,\n  \"fields\":
[\n    {\n      \"column\": \"title\",\n      \"properties\": {\n
\"dtype\": \"string\",\n        \"num_unique_values\": 62308,\n
\"samples\": [\n          \"About 146,000 Rohingya have fled Myanmar
violence to Bangladesh\",\n          \"Franklin Graham: The media
didn\\u2019t understand the \\u2018God-factor\\u2019 in Trump\\u2019s
win\",\n          \"T\\u00fcrkei: Kritischer Journalist interviewt
Oppositionspolitiker in gemeinsamer Gef\\u00e4ngniszelle\"\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"text\",\n      \"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 62200,\n        \"samples\": [\n
\"Analysis from the Brookings Institution says that 15 million people
could lose coverage under Speaker Paul Ryan\\u2019s American Health
Care Act. [The Congressional Budget Office (CBO) has yet to score the
Ryan plan, although the Brookings Institution\\u2019s report does not
bode well for the House leadership\\u2019s bill.  The Ryan plan
reports, \\u201cWe conclude that CBO\\u2019s analysis will likely
estimate that at least 15 million people will lose coverage under the
American Health Care Act (AHCA) by the end of the   scoring window.
Estimates could be higher, but it\\u2019s [sic] is unlikely they will
be significantly lower. \\u201d The Kaiser Family Foundation states
that the Ryan plan\\u2019s tax credits aid wealthier Americans more
than   individuals. Older Americans would face comparatively less aid
through tax credits than younger counterparts. American Medical
Association CEO James Madara also criticized the Republican
```

leadership\\u2019s tax credits, saying, \\u201cWe believe credits inversely related to income, rather than age as proposed in the committee\\u2019s legislation, not only result in greater numbers of people insured but are a more efficient use of resources. \\u201d Ways and Means Chairman Kevin Brady ( ) has maintained that tax credits help everyone get covered. He said, \\u201cIt covers more people, because it applies to those who don\\u2019t have a tax liability, and it\\u2019s advanceable, so it\\u2019s available today. \\u201d Avik Roy, a health expert at Forbes, argued that tax credits in the Ryan plan will \\u201cprice many poor and vulnerable people out of the health insurance market. \\u201d Michael Cannon, the Cato Institute\\u2019s director of health policy studies, believes that the GOP\\u2019s Obamacare replacement bill will force the poor to lose their coverage. He lamented, \\u201cIf the tax credits (read: subsidies) for Americans are less than under Obamacare, many more patients will lose coverage. Premiums will continue to rise. Republicans will take the blame for all of it, because they will have failed to repeal Obamacare, or learn its lessons, when they had the chance. \\u201d\",\n   \"At least six members of the Super New England Patriots have said they will not visit the White House for the traditional meeting that championship teams have with the president. Running back LeGarrette Blount, defensive end Chris Long and defensive tackle Alan Branch on Thursday became the latest to announce that they would skip the visit. Long posted on Twitter that he \\u201cplanned on skipping\\u201d in response to a column in The Daily News that urged him to skip. Blount revealed his decision on Fox Sports in a radio interview on \\u201cThe Rich Eisen Show\\u201d later in the afternoon. \\u201cI just don\\u2019t feel welcome in that house,\\u201d Blount said. \\u201cI\\u2019m going to just leave it at that. \\u201d Branch told Sirius XM Radio that he planned to spend time with family. A day earlier, the Pro Bowl linebacker Dont\\u2019a Hightower bowed out when he told ESPN, \\u201cBeen there, done that,\\u201d having visited with a championship Alabama team. Tight end Martellus Bennett told reporters after the Super Bowl that he would not go: \\u201cIt is what it is,\\u201d he said. \\u201cPeople know how I feel about it. Just follow me on Twitter. \\u201d The outspoken Bennett had joked that he might move to outer space after Donald J. Trump was elected. The Pro Bowl safety Devin McCourty, a team captain, told Time magazine: \\u201cBasic reason for me is I don\\u2019t feel accepted in the White House. With the president having so many strong opinions and prejudices, I believe certain people might feel accepted there while others won\\u2019t. \\u201d Bennett and McCourty were in the news for their activism last fall, when they raised their fists in protest during the national anthem for one game. At the time, athletes in various sports were protesting racial oppression in the country. The number of Patriots absences may well increase. Running back James White said he had not made up his mind whether to go. \\u201cI\\u2019ll wait till the time comes and decide then,\\u201d he said. No date has been scheduled for this year\\u2019s ceremony honoring the

Patriots. The ceremony for last year\\u2019s champions, the Broncos, took place in June. Perhaps no other team has as close an association with Trump as the Patriots. Just before the election, Trump claimed that he had the support of Patriots quarterback Tom Brady and Coach Bill Belichick. Brady, who displayed a \\u201cMake America Great Again\\u201d cap in his locker during the campaign, never explicitly endorsed Trump, but he spoke favorably of him and they have socialized. Trump also cited a supportive letter he had received from Belichick, and several news media accounts confirmed the letter was authentic. The team\\u2019s owner, Robert K. Kraft, has said he has considered Trump a longtime friend. Brady did not attend his team\\u2019s visit with former President Barack Obama at the White House, in 2015, citing family issues. But some athletes who have skipped the trip over the years have explicitly given politics as a reason. Bruins goalie Tim Thomas declined to visit the Obama White House with his teammates in January 2012, saying in a statement: \\u201cI believe the federal government has grown out of control, threatening the rights, liberties and property of the people. \\u201cBecause I believe this, today I exercised my right as a free citizen, and did not visit the White House. \\u201d Baltimore Ravens center Matt Birk cited his opposition to abortion as the reason for skipping a 2013 visit. A number of other athletes have skipped the ceremony while citing scheduling conflicts or family commitments. Presidents for years have invited sports figures to the White House, but the tradition of honoring championships teams there solidified under Ronald Reagan. Major professional champions and many college champions stop by for a presentation and a photo opportunity. Six Patriots, so far, won\\u2019t make the trip this year.\",\n   \"SAN FRANCISCO \\u2014 By mastering some tough computer chip technology and then pulling off an audacious business strategy based on selling its smartphone knowledge, Qualcomm has made billions of dollars and seized influence in the tech industry. Some days that is a real drag. On Friday, Apple, its longtime partner, sued Qualcomm over what it said was $1 billion in withheld rebates. In the lawsuit, filed in Federal District Court for the Southern District of California, in San Diego, Apple said the money had been promised in conjunction with an agreement not to buy chips from other suppliers or to divulge Qualcomm\\u2019s intellectual property licensing practices. The suit seeks the rebates, among other things. Apple sued three days after the Federal Trade Commission accused Qualcomm of using anticompetitive practices to guarantee its high royalty payments for advanced wireless technology. The commission cited Qualcomm\\u2019s deals with Taiwanese companies that manufacture Apple iPhones over semiconductors it sells for the iPhone. In its complaint, the commission described Qualcomm as \\u201cthe world\\u2019s dominant supplier\\u201d of semiconductors that manage smartphone communications. The commission said Qualcomm had obtained \\u201celevated royalties\\u201d for its patented intellectual property on wireless communications. Other smartphone chip makers have to pay Qualcomm for that intellectual property, too. The commission

and Apple complaints follow several other problems, including a $975 million fine by Chinese regulators in 2015 and activist shareholder complaints later that year that forced layoffs at Qualcomm. Last month, Qualcomm was fined $850 million in South Korea for unfair patent licensing. When Apple provided information to South Korean regulators in that case, and sought competing chips from Intel, Qualcomm refused to pay Apple its promised money, Apple said. Qualcomm has vowed to fight the South Korean charge and the F. T. C. suit. \\u201cApple has intentionally mischaracterized our agreements and negotiations,\\u201d Qualcomm said in a statement, adding that it would fight that lawsuit, too. Few expect Qualcomm to back down. It was shaped by its founders to be brainy, combative and profitable. \\u201cThey are seasoned, and equipped to fight,\\u201d said Mark Hung, an analyst at Gartner. \\u201cBut when such a big company is so reliant on the way it sells its intellectual property, and under the microscope in various geographies, it\\u2019s tough. \\u201d Qualcomm, which is based in San Diego, far from Silicon Valley, was founded in 1985 by seven people, including Irwin Jacobs and Andrew Viterbi. The two were electrical engineering professors who had formed a military satellite communications company. Qualcomm\\u2019s engineers work at a campus between a park and the University of California, San Diego. They have earned thousands of patents, including for the airplane mode on smartphones and the way apps are downloaded from stores. Companies like Samsung and Apple need Qualcomm for the technology that enables things like wireless video for millions of people at once. Its major technology is spread spectrum, which harnesses lots of computation and clever radio engineering for use in the various functions of smartphones. Qualcomm\\u2019s radio technology is called CDMA, for code division multiple access, a way voice calls work more efficiently over cellphone networks. And the company\\u2019s influence grew over the last several years with the advent of or 3G, phones as people started downloading lots of mobile data as well as making calls. At the time, many thought Qualcomm\\u2019s success would be limited to that transition. Instead, Qualcomm innovated in 4G wireless for the new smartphones from Samsung and Apple that toppled Nokia and Motorola. In its last fiscal year, Qualcomm had revenue of $23. 6 billion and a profit of $5. 7 billion. Qualcomm\\u2019s ventures in phones and network gear helped promote CDMA but did not make much money because older companies did not need CDMA until the success of wireless began to tax the capacity of their networks. Qualcomm turned to Asian companies that wanted to get into wireless. \\u201cYou\\u2019d never have seen the Koreans or the Chinese in wireless if not for Qualcomm and CDMA,\\u201d said Clint McClellan, a employee of Qualcomm who now runs a wireless health business. \\u201cWe enabled those markets. \\u201d South Korea, in particular, made CDMA an industrial policy, which carried Qualcomm for years. More recently, China accounted for half of Qualcomm\\u2019s revenue. Apple\\u2019s Taiwanese manufacturers, which pay Qualcomm royalties for iPhones, are another big profit center. Qualcomm would not say what it charges for

its intellectual property, but analysts figure the fee usually amounts to 5 percent of the selling price of a phone \\u2014 at least it did, until the 2015 Chinese fine, which included an agreement that Chinese manufacturers pay only about half the old Qualcomm royalty. That, and problems collecting Chinese royalties even before the agreement, hurt Qualcomm\\u2019s share price. In 2015, Jana Partners, an activist hedge fund, stepped in and pressed for a breakup of the company into separate intellectual property and chip businesses. That did not happen, but Qualcomm cut about 15 percent of its staff, cut executive pay and gave Jana three board seats. The stock has recovered to about where it was just before the fine, but not to the heights it reached before the problems. By helping to create today\\u2019s smartphone giants, Qualcomm may have engineered its current woes. Qualcomm executives say they see a connection between the China discounts, which Qualcomm had hoped would be limited to that country, and the latest regulatory attacks. \\u201cWe believe Samsung had a major role to play in the Korean decision,\\u201d said Don Rosenberg, a lawyer for Qualcomm. The regulators, he said, \\u201cwere prodded and misled by commercial interests. \\u201d In an email, Jee, a spokeswoman for Samsung, said it \\u201cwas one of many multinational corporations\\u201d that responded to questions from South Korean regulators. Qualcomm\\u2019s lawyer figures the Korean case may take years to move through the courts. And recent events, including the impeachment of South Korea\\u2019s president and a bribery scandal at Samsung, could complicate, or eliminate, Qualcomm\\u2019s regulatory issues there. Additionally, the F. T. C. has just three of its normal five commissioners. One of those, Maureen Ohlhausen, a Republican, voted against the suit. She may be named chairwoman when the current chairwoman, Edith Ramirez, a Democrat, steps down on Feb. 10. Neither the F. T. C. nor Apple would comment on the United States case against Qualcomm. \\u201cIt\\u2019s disappointing when our partners want to pay less after we\\u2019ve contributed to their success,\\u201d said Derek Aberle, Qualcomm\\u2019s president. Now it must get out ahead of other technology inventors in 5G, possibly while paying billions in fines and dealing with a painful restructuring and fraying relationships with its biggest customers. \\u201cWe\\u2019re not sitting still\\u201d while the problems are sorted, Mr. Aberle said. \\u201cYou have to invest, and invent cool stuff that will change the way people live their lives. \\u201d In other words, create unique technology so people will keep paying for Qualcomm\\u2019s expensive intellectual property.\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n \"column\": \"label\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0,\n   \"min\": 0,\n \"max\": 1,\n   \"num_unique_values\": 2,\n   \"samples\": [\n   0,\n   1\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe","variable_name":"df"}

```
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess_text(text):

    if pd.isna(text):
        return ""

    # 1. Lowercase
    text = text.lower()

    # 2. Remove URLs
    text = re.sub(r'http\S+|www\S+', '', text)

    # 3. Remove HTML tags
    text = re.sub(r'<.*?>', '', text)

    # 4. Remove @mentions
    text = re.sub(r'@\w+', '', text)

    # 5. Remove hashtags
    text = re.sub(r'#\w+', '', text)

    # 6. Remove punctuation
    text = text.translate(str.maketrans('', '', string.punctuation))

    # 7. Remove numbers
    text = re.sub(r'\d+', '', text)

    # 8. Normalize whitespace
    text = re.sub(r'\s+', ' ', text).strip()

    # 9. Remove stopwords + Lemmatization
    words = [
        lemmatizer.lemmatize(word)
        for word in text.split()
        if word not in stop_words
    ]

    return " ".join(words)

sample = df.loc[0, "text"]

print("Original:\n")
print(sample[:600])

print("\n-----------------------------\n")

print("Processed:\n")
print(preprocess_text(sample)[:600])
```

```
Original:

No comment is expected from Barack Obama Members of the #FYF911 or
#FukYoFlag and #BlackLivesMatter movements called for the lynching and
hanging of white people and cops. They encouraged others on a radio
show Tuesday night to  turn the tide  and kill white people and cops
to send a message about the killing of black people in America.One of
the F***YoFlag organizers is called  Sunshine.  She has a radio blog
show hosted from Texas called,  Sunshine s F***ing Opinion Radio Show.
A snapshot of her #FYF911 @LOLatWhiteFear Twitter page at 9:53 p.m.
shows that she was urging supporters to  Call n

----------------------------

Processed:

comment expected barack obama member movement called lynching hanging
white people cop encouraged others radio show tuesday night turn tide
kill white people cop send message killing black people americaone
fyoflag organizer called sunshine radio blog show hosted texas called
sunshine fing opinion radio show snapshot twitter page pm show urging
supporter call tonight continue dismantle illusion white snapshot
twitter radio call invite radio show aired pm eastern standard
timeduring show caller clearly call lynching killing white peoplea
minute clip radio show heard provided breitbart texas som
```

```
from tqdm import tqdm

tqdm.pandas()

df["clean_title"] = df["title"].progress_apply(preprocess_text)
df["clean_text"] = df["text"].progress_apply(preprocess_text)
```

```
100%|██████████| 71537/71537 [00:06<00:00, 11900.90it/s]
100%|██████████| 71537/71537 [02:41<00:00, 443.91it/s]
```

```
# Experiment 1
headline_df = df[["clean_title", "label"]].copy()

# Experiment 2
content_df = df[["clean_text", "label"]].copy()

# Experiment 3
combined_df = pd.DataFrame()

combined_df["text"] = df["clean_title"] + " " + df["clean_text"]

combined_df["label"] = df["label"]

headline_df.to_csv("Headline_Only.csv", index=False)
```

```
content_df.to_csv("Content_Only.csv", index=False)

combined_df.to_csv("Headline_Content.csv", index=False)

print("All datasets saved successfully!")
```

```
All datasets saved successfully!
```

```
from sklearn.model_selection import train_test_split

X = headline_df["clean_title"]
y = headline_df["label"]

# 20% test
X_temp, X_test, y_temp, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    stratify=y,
    random_state=42
)

# 20% validation, 60% training
X_train, X_val, y_train, y_val = train_test_split(
    X_temp,
    y_temp,
    test_size=0.25,
    stratify=y_temp,
    random_state=42
)

print("Training samples :", len(X_train))
print("Validation samples:", len(X_val))
print("Test samples      :", len(X_test))
```

```
Training samples : 42921
Validation samples: 14308
Test samples      : 14308
```

```
print("NaN in X_train:", X_train.isna().sum())
print("NaN in X_val:", X_val.isna().sum())
print("NaN in X_test:", X_test.isna().sum())
```

```
NaN in X_train: 0
NaN in X_val: 0
NaN in X_test: 0
```

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer

# Make a clean copy
```

```
headline_df = pd.read_csv("Headline_Only.csv")

# Remove missing text or labels
headline_df = headline_df.dropna(
    subset=["clean_title", "label"]
).copy()

# Ensure text is actually string
headline_df["clean_title"] = headline_df["clean_title"].astype(str)

# Separate input and target
X = headline_df["clean_title"]
y = headline_df["label"]

# -------------------------
# 60% Train / 20% Validation / 20% Test
# -------------------------

X_temp, X_test, y_temp, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    stratify=y,
    random_state=42
)

X_train, X_val, y_train, y_val = train_test_split(
    X_temp,
    y_temp,
    test_size=0.25,
    stratify=y_temp,
    random_state=42
)

print("Dataset:", len(headline_df))
print("Training:", len(X_train))
print("Validation:", len(X_val))
print("Test:", len(X_test))

# Check for missing values
print("\nMissing values:")
print("Train:", X_train.isna().sum())
print("Validation:", X_val.isna().sum())
print("Test:", X_test.isna().sum())

# -------------------------
# TF-IDF
# -------------------------

tfidf = TfidfVectorizer(
```

```
    max_features=1000,
    ngram_range=(1, 2)
)

# Fit ONLY on training data
X_train_tfidf = tfidf.fit_transform(X_train)

# Transform validation and test
X_val_tfidf = tfidf.transform(X_val)
X_test_tfidf = tfidf.transform(X_test)

print("\nTF-IDF shapes:")
print("Training:", X_train_tfidf.shape)
print("Validation:", X_val_tfidf.shape)
print("Test:", X_test_tfidf.shape)
```

```
Dataset: 71524
Training: 42914
Validation: 14305
Test: 14305

Missing values:
Train: 0
Validation: 0
Test: 0

TF-IDF shapes:
Training: (42914, 1000)
Validation: (14305, 1000)
Test: (14305, 1000)
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score
)

# Create Logistic Regression model
lr_model = LogisticRegression(
    max_iter=1000,
    random_state=42
)

# Train
lr_model.fit(X_train_tfidf, y_train)

# Predictions
```

```python
y_val_pred = lr_model.predict(X_val_tfidf)
y_test_pred = lr_model.predict(X_test_tfidf)

# Probabilities for ROC-AUC
y_val_prob = lr_model.predict_proba(X_val_tfidf)[:, 1]
y_test_prob = lr_model.predict_proba(X_test_tfidf)[:, 1]

# Validation metrics
print("VALIDATION RESULTS")
print("------------------")
print("Accuracy :", accuracy_score(y_val, y_val_pred))
print("Precision:", precision_score(y_val, y_val_pred))
print("Recall   :", recall_score(y_val, y_val_pred))
print("F1 Score :", f1_score(y_val, y_val_pred))
print("ROC-AUC  :", roc_auc_score(y_val, y_val_prob))

# Test metrics
print("\nTEST RESULTS")
print("------------------")
print("Accuracy :", accuracy_score(y_test, y_test_pred))
print("Precision:", precision_score(y_test, y_test_pred))
print("Recall   :", recall_score(y_test, y_test_pred))
print("F1 Score :", f1_score(y_test, y_test_pred))
print("ROC-AUC  :", roc_auc_score(y_test, y_test_prob))
```

```
VALIDATION RESULTS
------------------
Accuracy : 0.8673191191890948
Precision: 0.8655746581832949
Recall   : 0.8760104123852582
F1 Score : 0.870761269236007
ROC-AUC  : 0.9411898700571646

TEST RESULTS
------------------
Accuracy : 0.8684376092275429
Precision: 0.8687542545949626
Recall   : 0.87422934648582
F1 Score : 0.8714832013111172
ROC-AUC  : 0.9444384702725009
```

```python
from sklearn.ensemble import RandomForestClassifier

# Create Random Forest model
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)
```

```python
# Train
rf_model.fit(X_train_tfidf, y_train)

# Predictions
y_val_pred_rf = rf_model.predict(X_val_tfidf)
y_test_pred_rf = rf_model.predict(X_test_tfidf)

# Probabilities for ROC-AUC
y_val_prob_rf = rf_model.predict_proba(X_val_tfidf)[:, 1]
y_test_prob_rf = rf_model.predict_proba(X_test_tfidf)[:, 1]

# Validation
print("VALIDATION RESULTS")
print("------------------")
print("Accuracy :", accuracy_score(y_val, y_val_pred_rf))
print("Precision:", precision_score(y_val, y_val_pred_rf))
print("Recall   :", recall_score(y_val, y_val_pred_rf))
print("F1 Score :", f1_score(y_val, y_val_pred_rf))
print("ROC-AUC  :", roc_auc_score(y_val, y_val_prob_rf))

# Test
print("\nTEST RESULTS")
print("------------------")
print("Accuracy :", accuracy_score(y_test, y_test_pred_rf))
print("Precision:", precision_score(y_test, y_test_pred_rf))
print("Recall   :", recall_score(y_test, y_test_pred_rf))
print("F1 Score :", f1_score(y_test, y_test_pred_rf))
print("ROC-AUC  :", roc_auc_score(y_test, y_test_prob_rf))
```

```
VALIDATION RESULTS
------------------
Accuracy : 0.8771059070255156
Precision: 0.8753556428668202
Recall   : 0.8851897520208247
F1 Score : 0.8802452316076295
ROC-AUC  : 0.9460787275792064

TEST RESULTS
------------------
Accuracy : 0.8793428871024117
Precision: 0.8765031752465883
Recall   : 0.8887518838197014
F1 Score : 0.8825850340136054
ROC-AUC  : 0.9482994964447712
```

```python
from lightgbm import LGBMClassifier

# Create LightGBM model
lgbm_model = LGBMClassifier(
    n_estimators=100,
```

```
    learning_rate=0.1,
    random_state=42,
    n_jobs=-1,
    verbosity=-1
)

# Train
lgbm_model.fit(X_train_tfidf, y_train)

# Predictions
y_val_pred_lgbm = lgbm_model.predict(X_val_tfidf)
y_test_pred_lgbm = lgbm_model.predict(X_test_tfidf)

# Probabilities
y_val_prob_lgbm = lgbm_model.predict_proba(X_val_tfidf)[:, 1]
y_test_prob_lgbm = lgbm_model.predict_proba(X_test_tfidf)[:, 1]

# Validation
print("VALIDATION RESULTS")
print("------------------")
print("Accuracy :", accuracy_score(y_val, y_val_pred_lgbm))
print("Precision:", precision_score(y_val, y_val_pred_lgbm))
print("Recall   :", recall_score(y_val, y_val_pred_lgbm))
print("F1 Score :", f1_score(y_val, y_val_pred_lgbm))
print("ROC-AUC  :", roc_auc_score(y_val, y_val_prob_lgbm))

# Test
print("\nTEST RESULTS")
print("------------------")
print("Accuracy :", accuracy_score(y_test, y_test_pred_lgbm))
print("Precision:", precision_score(y_test, y_test_pred_lgbm))
print("Recall   :", recall_score(y_test, y_test_pred_lgbm))
print("F1 Score :", f1_score(y_test, y_test_pred_lgbm))
print("ROC-AUC  :", roc_auc_score(y_test, y_test_prob_lgbm))
```

```
/usr/local/lib/python3.13/dist-packages/sklearn/utils/
validation.py:2739: UserWarning: X does not have valid feature names,
but LGBMClassifier was fitted with feature names
  warnings.warn(
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:27
39: UserWarning: X does not have valid feature names, but
LGBMClassifier was fitted with feature names
  warnings.warn(
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:27
39: UserWarning: X does not have valid feature names, but
LGBMClassifier was fitted with feature names
  warnings.warn(

VALIDATION RESULTS
------------------
```

```
Accuracy : 0.8614470464872422
Precision: 0.8455738983491485
Recall   : 0.8912179750650774
F1 Score : 0.8677961579509071
ROC-AUC  : 0.9399595621892135

TEST RESULTS
------------------
Accuracy : 0.8632645927997203
Precision: 0.8474444010924698
Recall   : 0.8927250308261405
F1 Score : 0.8694955964771818
ROC-AUC  : 0.9418497667256965
```

```python
# ==============================
# EXPERIMENT 2 — CONTENT ONLY
# ==============================

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer

# Load Content-Only dataset
content_df = pd.read_csv("Content_Only.csv")

# Remove missing values
content_df = content_df.dropna(
    subset=["clean_text", "label"]
).copy()

# Make sure text is string
content_df["clean_text"] = content_df["clean_text"].astype(str)

# Separate features and target
X_content = content_df["clean_text"]
y_content = content_df["label"]

# ------------------------------
# 60% Train / 20% Validation / 20% Test
# ------------------------------

X_temp_c, X_test_c, y_temp_c, y_test_c = train_test_split(
    X_content,
    y_content,
    test_size=0.20,
    stratify=y_content,
    random_state=42
)

X_train_c, X_val_c, y_train_c, y_val_c = train_test_split(
    X_temp_c,
```

```
    y_temp_c,
    test_size=0.25,
    stratify=y_temp_c,
    random_state=42
)

print("Dataset:", len(content_df))
print("Training:", len(X_train_c))
print("Validation:", len(X_val_c))
print("Test:", len(X_test_c))

# ----------------------------
# TF-IDF
# ----------------------------

tfidf_content = TfidfVectorizer(
    max_features=1000,
    ngram_range=(1, 2)
)

# Fit ONLY on training content
X_train_content_tfidf = tfidf_content.fit_transform(X_train_c)

# Transform validation and test
X_val_content_tfidf = tfidf_content.transform(X_val_c)
X_test_content_tfidf = tfidf_content.transform(X_test_c)

print("\nTF-IDF shapes:")
print("Training:", X_train_content_tfidf.shape)
print("Validation:", X_val_content_tfidf.shape)
print("Test:", X_test_content_tfidf.shape)
```

```
Dataset: 70705
Training: 42423
Validation: 14141
Test: 14141

TF-IDF shapes:
Training: (42423, 1000)
Validation: (14141, 1000)
Test: (14141, 1000)
```

```
# ==============================
# Logistic Regression — Content Only
# ==============================

lr_content = LogisticRegression(
    max_iter=1000,
    random_state=42
)
```

```python
# Train
lr_content.fit(X_train_content_tfidf, y_train_c)

# Predictions
y_val_pred_lr_c = lr_content.predict(X_val_content_tfidf)
y_test_pred_lr_c = lr_content.predict(X_test_content_tfidf)

# Probabilities
y_val_prob_lr_c = lr_content.predict_proba(X_val_content_tfidf)[:, 1]
y_test_prob_lr_c = lr_content.predict_proba(X_test_content_tfidf)[:,
1]

# Validation results
print("VALIDATION RESULTS")
print("------------------")
print("Accuracy :", accuracy_score(y_val_c, y_val_pred_lr_c))
print("Precision:", precision_score(y_val_c, y_val_pred_lr_c))
print("Recall   :", recall_score(y_val_c, y_val_pred_lr_c))
print("F1 Score :", f1_score(y_val_c, y_val_pred_lr_c))
print("ROC-AUC  :", roc_auc_score(y_val_c, y_val_prob_lr_c))

# Test results
print("\nTEST RESULTS")
print("------------------")
print("Accuracy :", accuracy_score(y_test_c, y_test_pred_lr_c))
print("Precision:", precision_score(y_test_c, y_test_pred_lr_c))
print("Recall   :", recall_score(y_test_c, y_test_pred_lr_c))
print("F1 Score :", f1_score(y_test_c, y_test_pred_lr_c))
print("ROC-AUC  :", roc_auc_score(y_test_c, y_test_prob_lr_c))
```

```
VALIDATION RESULTS
------------------
Accuracy : 0.9285764797397638
Precision: 0.9245910729137787
Recall   : 0.9346973094170403
F1 Score : 0.929616724738676
ROC-AUC  : 0.9800352506857689

TEST RESULTS
------------------
Accuracy : 0.9331023265681352
Precision: 0.9322625698324022
Recall   : 0.9353979820627802
F1 Score : 0.9338276440962507
ROC-AUC  : 0.9815137849966231
```

```python
# ==============================
# Random Forest — Content Only
# ==============================
```

```python
rf_content = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)

# Train
rf_content.fit(X_train_content_tfidf, y_train_c)

# Predictions
y_val_pred_rf_c = rf_content.predict(X_val_content_tfidf)
y_test_pred_rf_c = rf_content.predict(X_test_content_tfidf)

# Probabilities
y_val_prob_rf_c = rf_content.predict_proba(X_val_content_tfidf)[:, 1]
y_test_prob_rf_c = rf_content.predict_proba(X_test_content_tfidf)[:,
1]

# Validation
print("VALIDATION RESULTS")
print("------------------")
print("Accuracy :", accuracy_score(y_val_c, y_val_pred_rf_c))
print("Precision:", precision_score(y_val_c, y_val_pred_rf_c))
print("Recall   :", recall_score(y_val_c, y_val_pred_rf_c))
print("F1 Score :", f1_score(y_val_c, y_val_pred_rf_c))
print("ROC-AUC  :", roc_auc_score(y_val_c, y_val_prob_rf_c))

# Test
print("\nTEST RESULTS")
print("------------------")
print("Accuracy :", accuracy_score(y_test_c, y_test_pred_rf_c))
print("Precision:", precision_score(y_test_c, y_test_pred_rf_c))
print("Recall   :", recall_score(y_test_c, y_test_pred_rf_c))
print("F1 Score :", f1_score(y_test_c, y_test_pred_rf_c))
print("ROC-AUC  :", roc_auc_score(y_test_c, y_test_prob_rf_c))
```

```
VALIDATION RESULTS
------------------
Accuracy : 0.943780496428824
Precision: 0.9298983050847458
Recall   : 0.961042600896861
F1 Score : 0.9452139756047137
ROC-AUC  : 0.9878687708651411

TEST RESULTS
------------------
Accuracy : 0.9459727034863163
Precision: 0.9363188167625308
Recall   : 0.9580997757847534
```

```
F1 Score : 0.947084083668098
ROC-AUC  : 0.9884707291876719

# ==============================
# LightGBM — Content Only
# ==============================

from lightgbm import LGBMClassifier

lgbm_content = LGBMClassifier(
    n_estimators=100,
    learning_rate=0.1,
    random_state=42,
    n_jobs=-1,
    verbosity=-1
)

# Train
lgbm_content.fit(X_train_content_tfidf, y_train_c)

# Predictions
y_val_pred_lgbm_c = lgbm_content.predict(X_val_content_tfidf)
y_test_pred_lgbm_c = lgbm_content.predict(X_test_content_tfidf)

# Probabilities
y_val_prob_lgbm_c = lgbm_content.predict_proba(X_val_content_tfidf)[:,
1]
y_test_prob_lgbm_c = lgbm_content.predict_proba(X_test_content_tfidf)
[:, 1]

# Validation
print("VALIDATION RESULTS")
print("------------------")
print("Accuracy :", accuracy_score(y_val_c, y_val_pred_lgbm_c))
print("Precision:", precision_score(y_val_c, y_val_pred_lgbm_c))
print("Recall   :", recall_score(y_val_c, y_val_pred_lgbm_c))
print("F1 Score :", f1_score(y_val_c, y_val_pred_lgbm_c))
print("ROC-AUC  :", roc_auc_score(y_val_c, y_val_prob_lgbm_c))

# Test
print("\nTEST RESULTS")
print("------------------")
print("Accuracy :", accuracy_score(y_test_c, y_test_pred_lgbm_c))
print("Precision:", precision_score(y_test_c, y_test_pred_lgbm_c))
print("Recall   :", recall_score(y_test_c, y_test_pred_lgbm_c))
print("F1 Score :", f1_score(y_test_c, y_test_pred_lgbm_c))
print("ROC-AUC  :", roc_auc_score(y_test_c, y_test_prob_lgbm_c))

/usr/local/lib/python3.13/dist-packages/sklearn/utils/
validation.py:2739: UserWarning: X does not have valid feature names,
```

```
but LGBMClassifier was fitted with feature names
  warnings.warn(
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:27
39: UserWarning: X does not have valid feature names, but
LGBMClassifier was fitted with feature names
  warnings.warn(
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:27
39: UserWarning: X does not have valid feature names, but
LGBMClassifier was fitted with feature names
  warnings.warn(

VALIDATION RESULTS
------------------
Accuracy : 0.9509228484548476
Precision: 0.9366865509761388
Recall   : 0.968189461883408
F1 Score : 0.9521775082690187
ROC-AUC  : 0.9895525257423429

TEST RESULTS
------------------
Accuracy : 0.95311505551234
Precision: 0.9429314356096893
Recall   : 0.9655269058295964
F1 Score : 0.9540954095409541
ROC-AUC  : 0.9899614465004177

# ============================================
# EXPERIMENT 3 — HEADLINE + CONTENT
# ============================================

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer

# Load combined dataset
combined_df = pd.read_csv("Headline_Content.csv")

# Remove missing values
combined_df = combined_df.dropna(
    subset=["text", "label"]
).copy()

combined_df["text"] = combined_df["text"].astype(str)

# Separate features and target
X_combined = combined_df["text"]
y_combined = combined_df["label"]

# 60% Train / 20% Validation / 20% Test
```

```python
X_temp_comb, X_test_comb, y_temp_comb, y_test_comb = train_test_split(
    X_combined,
    y_combined,
    test_size=0.20,
    stratify=y_combined,
    random_state=42
)

X_train_comb, X_val_comb, y_train_comb, y_val_comb = train_test_split(
    X_temp_comb,
    y_temp_comb,
    test_size=0.25,
    stratify=y_temp_comb,
    random_state=42
)

print("Dataset:", len(combined_df))
print("Training:", len(X_train_comb))
print("Validation:", len(X_val_comb))
print("Test:", len(X_test_comb))

# TF-IDF
tfidf_combined = TfidfVectorizer(
    max_features=1000,
    ngram_range=(1, 2)
)

# Fit ONLY on combined training data
X_train_comb_tfidf = tfidf_combined.fit_transform(X_train_comb)

# Transform validation and test
X_val_comb_tfidf = tfidf_combined.transform(X_val_comb)
X_test_comb_tfidf = tfidf_combined.transform(X_test_comb)

print("\nTF-IDF shapes:")
print("Training:", X_train_comb_tfidf.shape)
print("Validation:", X_val_comb_tfidf.shape)
print("Test:", X_test_comb_tfidf.shape)
```

```
Dataset: 71537
Training: 42921
Validation: 14308
Test: 14308

TF-IDF shapes:
Training: (42921, 1000)
Validation: (14308, 1000)
Test: (14308, 1000)
```

```python
# ========================================
# Logistic Regression — Headline + Content
# ========================================

lr_combined = LogisticRegression(
    max_iter=1000,
    random_state=42
)

lr_combined.fit(X_train_comb_tfidf, y_train_comb)

# Predictions
y_val_pred_lr_comb = lr_combined.predict(X_val_comb_tfidf)
y_test_pred_lr_comb = lr_combined.predict(X_test_comb_tfidf)

# Probabilities
y_val_prob_lr_comb = lr_combined.predict_proba(X_val_comb_tfidf)[:, 1]
y_test_prob_lr_comb = lr_combined.predict_proba(X_test_comb_tfidf)[:,
1]

print("VALIDATION RESULTS")
print("------------------")
print("Accuracy :", accuracy_score(y_val_comb, y_val_pred_lr_comb))
print("Precision:", precision_score(y_val_comb, y_val_pred_lr_comb))
print("Recall   :", recall_score(y_val_comb, y_val_pred_lr_comb))
print("F1 Score :", f1_score(y_val_comb, y_val_pred_lr_comb))
print("ROC-AUC  :", roc_auc_score(y_val_comb, y_val_prob_lr_comb))

print("\nTEST RESULTS")
print("------------------")
print("Accuracy :", accuracy_score(y_test_comb, y_test_pred_lr_comb))
print("Precision:", precision_score(y_test_comb, y_test_pred_lr_comb))
print("Recall   :", recall_score(y_test_comb, y_test_pred_lr_comb))
print("F1 Score :", f1_score(y_test_comb, y_test_pred_lr_comb))
print("ROC-AUC  :", roc_auc_score(y_test_comb, y_test_prob_lr_comb))
```

```
VALIDATION RESULTS
------------------
Accuracy : 0.938076600503215
Precision: 0.9371763423276097
Recall   : 0.9417967680087648
F1 Score : 0.9394808743169399
ROC-AUC  : 0.9844013657190814

TEST RESULTS
------------------
Accuracy : 0.9389851831143416
Precision: 0.9355100934832679
Recall   : 0.9456313338811284
```

```
F1 Score : 0.9405434856636927
ROC-AUC  : 0.9834584989678604

# ========================================
# Random Forest — Headline + Content
# ========================================

rf_combined = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)

# Train
rf_combined.fit(X_train_comb_tfidf, y_train_comb)

# Predictions
y_val_pred_rf_comb = rf_combined.predict(X_val_comb_tfidf)
y_test_pred_rf_comb = rf_combined.predict(X_test_comb_tfidf)

# Probabilities
y_val_prob_rf_comb = rf_combined.predict_proba(X_val_comb_tfidf)[:, 1]
y_test_prob_rf_comb = rf_combined.predict_proba(X_test_comb_tfidf)[:,
1]

# Validation
print("VALIDATION RESULTS")
print("------------------")
print("Accuracy :", accuracy_score(y_val_comb, y_val_pred_rf_comb))
print("Precision:", precision_score(y_val_comb, y_val_pred_rf_comb))
print("Recall   :", recall_score(y_val_comb, y_val_pred_rf_comb))
print("F1 Score :", f1_score(y_val_comb, y_val_pred_rf_comb))
print("ROC-AUC  :", roc_auc_score(y_val_comb, y_val_prob_rf_comb))

# Test
print("\nTEST RESULTS")
print("------------------")
print("Accuracy :", accuracy_score(y_test_comb, y_test_pred_rf_comb))
print("Precision:", precision_score(y_test_comb, y_test_pred_rf_comb))
print("Recall   :", recall_score(y_test_comb, y_test_pred_rf_comb))
print("F1 Score :", f1_score(y_test_comb, y_test_pred_rf_comb))
print("ROC-AUC  :", roc_auc_score(y_test_comb, y_test_prob_rf_comb))

VALIDATION RESULTS
------------------
Accuracy : 0.9558987978753145
Precision: 0.9455055429410979
Recall   : 0.969460421802246
F1 Score : 0.9573331530191358
ROC-AUC  : 0.9915315963864914
```

```
TEST RESULTS
------------------
Accuracy : 0.951006429969248
Precision: 0.9407130458005074
Recall   : 0.9648041632429472
F1 Score : 0.9526063146508011
ROC-AUC  : 0.9911955870982129

# ========================================
# LightGBM — Headline + Content
# ========================================

from lightgbm import LGBMClassifier

lgbm_combined = LGBMClassifier(
    n_estimators=100,
    learning_rate=0.1,
    random_state=42,
    n_jobs=-1,
    verbosity=-1
)

# Train
lgbm_combined.fit(X_train_comb_tfidf, y_train_comb)

# Predictions
y_val_pred_lgbm_comb = lgbm_combined.predict(X_val_comb_tfidf)
y_test_pred_lgbm_comb = lgbm_combined.predict(X_test_comb_tfidf)

# Probabilities
y_val_prob_lgbm_comb = lgbm_combined.predict_proba(X_val_comb_tfidf)
[:, 1]
y_test_prob_lgbm_comb = lgbm_combined.predict_proba(X_test_comb_tfidf)
[:, 1]

# Validation
print("VALIDATION RESULTS")
print("------------------")
print("Accuracy :", accuracy_score(y_val_comb, y_val_pred_lgbm_comb))
print("Precision:", precision_score(y_val_comb, y_val_pred_lgbm_comb))
print("Recall   :", recall_score(y_val_comb, y_val_pred_lgbm_comb))
print("F1 Score :", f1_score(y_val_comb, y_val_pred_lgbm_comb))
print("ROC-AUC  :", roc_auc_score(y_val_comb, y_val_prob_lgbm_comb))

# Test
print("\nTEST RESULTS")
print("------------------")
print("Accuracy :", accuracy_score(y_test_comb,
y_test_pred_lgbm_comb))
```

```
print("Precision:", precision_score(y_test_comb,
y_test_pred_lgbm_comb))
print("Recall   :", recall_score(y_test_comb, y_test_pred_lgbm_comb))
print("F1 Score :", f1_score(y_test_comb, y_test_pred_lgbm_comb))
print("ROC-AUC  :", roc_auc_score(y_test_comb, y_test_prob_lgbm_comb))
```

```
/usr/local/lib/python3.13/dist-packages/sklearn/utils/
validation.py:2739: UserWarning: X does not have valid feature names,
but LGBMClassifier was fitted with feature names
  warnings.warn(
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:27
39: UserWarning: X does not have valid feature names, but
LGBMClassifier was fitted with feature names
  warnings.warn(
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:27
39: UserWarning: X does not have valid feature names, but
LGBMClassifier was fitted with feature names
  warnings.warn(

VALIDATION RESULTS
------------------
Accuracy : 0.9619793122728544
Precision: 0.9530705282917672
Recall   : 0.9734319364557655
F1 Score : 0.9631436314363143
ROC-AUC  : 0.9929349597672394

TEST RESULTS
------------------
Accuracy : 0.9596729102599945
Precision: 0.9497124515179884
Recall   : 0.9724732949876747
F1 Score : 0.9609581162460248
ROC-AUC  : 0.9926837175913622
```

```
# ========================================
# GRU — HEADLINE ONLY
# Step 1: Tokenization
# ========================================

from tensorflow.keras.preprocessing.text import Tokenizer

# Parameters
MAX_WORDS = 10000

# Create tokenizer
tokenizer_headline = Tokenizer(
    num_words=MAX_WORDS,
    oov_token="<OOV>"
)
```

```
# IMPORTANT:
# Fit tokenizer ONLY on training text
tokenizer_headline.fit_on_texts(X_train)

# Convert text → integer sequences
X_train_seq = tokenizer_headline.texts_to_sequences(X_train)
X_val_seq = tokenizer_headline.texts_to_sequences(X_val)
X_test_seq = tokenizer_headline.texts_to_sequences(X_test)

print("Vocabulary size:", len(tokenizer_headline.word_index))
print("Example sequence:")
print(X_train_seq[0])
```

```
Vocabulary size: 33338
Example sequence:
[662, 235, 1668, 305, 807, 66, 4092, 66, 390]
```

```
# ==========================================
# GRU — HEADLINE ONLY
# Step 2: Padding
# ==========================================

from tensorflow.keras.preprocessing.sequence import pad_sequences

MAX_LEN = 50

X_train_pad = pad_sequences(
    X_train_seq,
    maxlen=MAX_LEN,
    padding="post",
    truncating="post"
)

X_val_pad = pad_sequences(
    X_val_seq,
    maxlen=MAX_LEN,
    padding="post",
    truncating="post"
)

X_test_pad = pad_sequences(
    X_test_seq,
    maxlen=MAX_LEN,
    padding="post",
    truncating="post"
)

print("Training shape:", X_train_pad.shape)
print("Validation shape:", X_val_pad.shape)
print("Test shape:", X_test_pad.shape)
```

```
print("\nExample padded sequence:")
print(X_train_pad[0])
```

```
Training shape: (42914, 50)
Validation shape: (14305, 50)
Test shape: (14305, 50)

Example padded sequence:
[ 662  235 1668  305  807   66 4092   66  390    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0]
```

```
# ==========================================
# GRU — HEADLINE ONLY
# Step 3: Build Model
# ==========================================

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, GRU, Dense, Dropout

model_gru_headline = Sequential([
    Embedding(
        input_dim=MAX_WORDS,
        output_dim=128,
        input_length=MAX_LEN
    ),

    GRU(64),

    Dropout(0.3),

    Dense(1, activation="sigmoid")
])

model_gru_headline.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

model_gru_headline.summary()
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/
embedding.py:100: UserWarning: Argument `input_length` is deprecated.
Just remove it.
  warnings.warn(

Model: "sequential"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━
┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━
┩
│ embedding (Embedding)                │ ?                           │   0
(unbuilt) │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ gru (GRU)                            │ ?                           │   0
(unbuilt) │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ dropout (Dropout)                    │ ?                           │
0 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ dense (Dense)                        │ ?                           │   0
(unbuilt) │
└──────────────────────────────────────┴─────────────────────────────┴─────────────────
┘

 Total params: 0 (0.00 B)

 Trainable params: 0 (0.00 B)

 Non-trainable params: 0 (0.00 B)
```

```python
# ==========================================
# GRU — HEADLINE ONLY
# Step 3: Build Model
# ==========================================

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Embedding, GRU, Dense,
Dropout

model_gru_headline = Sequential([
    Input(shape=(MAX_LEN,)),

    Embedding(
        input_dim=MAX_WORDS,
        output_dim=128
    ),

    GRU(64),

    Dropout(0.3),
```

```
    Dense(1, activation="sigmoid")
])

model_gru_headline.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

model_gru_headline.summary()
```

```
Model: "sequential_1"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━┓
┃ Layer (type)                         ┃ Output Shape                ┃         Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━┩
│ embedding_1 (Embedding)              │ (None, 50, 128)             │       1,280,000 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ gru_1 (GRU)                          │ (None, 64)                  │          37,248 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ dropout_1 (Dropout)                  │ (None, 64)                  │               0 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ dense_1 (Dense)                      │ (None, 1)                   │              65 │
└──────────────────────────────────────┴─────────────────────────────┴─────────────────┘
```

```
 Total params: 1,317,313 (5.03 MB)

 Trainable params: 1,317,313 (5.03 MB)

 Non-trainable params: 0 (0.00 B)
```

```
# ========================================
# GRU — HEADLINE ONLY
# Step 4: Training
# ========================================

from tensorflow.keras.callbacks import EarlyStopping

early_stop = EarlyStopping(
    monitor="val_loss",
```

```
    patience=2,
    restore_best_weights=True
)

history_gru_headline = model_gru_headline.fit(
    X_train_pad,
    y_train,
    validation_data=(X_val_pad, y_val),
    epochs=10,
    batch_size=128,
    callbacks=[early_stop],
    verbose=1
)
```

```
Epoch 1/10
336/336 ━━━━━━━━━━━━━━━━━━━━ 43s 122ms/step - accuracy: 0.5020 - loss:
0.6936 - val_accuracy: 0.5102 - val_loss: 0.6930
Epoch 2/10
336/336 ━━━━━━━━━━━━━━━━━━━━ 41s 121ms/step - accuracy: 0.5070 - loss:
0.6931 - val_accuracy: 0.5102 - val_loss: 0.6930
Epoch 3/10
336/336 ━━━━━━━━━━━━━━━━━━━━ 41s 121ms/step - accuracy: 0.5059 - loss:
0.6932 - val_accuracy: 0.5102 - val_loss: 0.6931
```

```
# ========================================
# GRU DEBUG CHECK
# ========================================

import numpy as np

print("X_train shape:", X_train_pad.shape)
print("X_val shape:", X_val_pad.shape)

print("\nLabel distribution:")
print("Training:")
print(np.unique(y_train, return_counts=True))

print("Validation:")
print(np.unique(y_val, return_counts=True))

print("\nNon-zero tokens:")
print("Training:", np.count_nonzero(X_train_pad))
print("Validation:", np.count_nonzero(X_val_pad))

print("\nExample sequences:")
for i in range(3):
    print(X_train_pad[i])
    print("Label:", y_train.iloc[i] if hasattr(y_train, "iloc") else
y_train[i])
```

```
X_train shape: (42914, 50)
X_val shape: (14305, 50)

Label distribution:
Training:
(array([0, 1]), array([21016, 21898]))
Validation:
(array([0, 1]), array([7006, 7299]))

Non-zero tokens:
Training: 387344
Validation: 129098

Example sequences:
[ 662  235 1668  305  807   66 4092   66  390    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0]
Label: 1
[1882 2559   42    1 1127 1628 3871 1075   30 1883    4    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0]
Label: 1
[263  49   2 391 264 241 227 508 397  42   0   0   0   0   0   0   0
0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
0
   0   0   0   0   0   0   0   0   0   0   0   0   0   0]
Label: 1
```

```python
# ==========================================
# GRU LEARNING DIAGNOSTIC
# Can the model overfit a small dataset?
# ==========================================

# Take only 1000 training samples
X_debug = X_train_pad[:1000]
y_debug = y_train.iloc[:1000].values

# Create a fresh small GRU
debug_model = Sequential([
    Input(shape=(MAX_LEN,)),

    Embedding(
        input_dim=MAX_WORDS,
        output_dim=128,
        mask_zero=True
    ),
```

```
    GRU(64),

    Dense(1, activation="sigmoid")
])

debug_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

debug_history = debug_model.fit(
    X_debug,
    y_debug,
    epochs=10,
    batch_size=32,
    verbose=1
)
```

```
Epoch 1/10
32/32 ━━━━━━━━━━━━━━━━━━━━ 4s 46ms/step - accuracy: 0.7040 - loss:
0.6345
Epoch 2/10
32/32 ━━━━━━━━━━━━━━━━━━━━ 3s 45ms/step - accuracy: 0.9130 - loss:
0.2850
Epoch 3/10
32/32 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 0.9810 - loss:
0.0680
Epoch 4/10
32/32 ━━━━━━━━━━━━━━━━━━━━ 3s 60ms/step - accuracy: 0.9970 - loss:
0.0188
Epoch 5/10
32/32 ━━━━━━━━━━━━━━━━━━━━ 3s 79ms/step - accuracy: 0.9990 - loss:
0.0078
Epoch 6/10
32/32 ━━━━━━━━━━━━━━━━━━━━ 2s 45ms/step - accuracy: 1.0000 - loss:
0.0036
Epoch 7/10
32/32 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 1.0000 - loss:
0.0020
Epoch 8/10
32/32 ━━━━━━━━━━━━━━━━━━━━ 2s 48ms/step - accuracy: 1.0000 - loss:
0.0012
Epoch 9/10
32/32 ━━━━━━━━━━━━━━━━━━━━ 1s 46ms/step - accuracy: 1.0000 - loss:
9.0877e-04
Epoch 10/10
32/32 ━━━━━━━━━━━━━━━━━━━━ 3s 45ms/step - accuracy: 1.0000 - loss:
7.1622e-04
```

```python
# ========================================
# GRU — HEADLINE ONLY
# Final Training Model
# ========================================

model_gru_headline = Sequential([
    Input(shape=(MAX_LEN,)),

    Embedding(
        input_dim=MAX_WORDS,
        output_dim=128,
        mask_zero=True
    ),

    GRU(64),

    Dropout(0.3),

    Dense(1, activation="sigmoid")
])

model_gru_headline.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

model_gru_headline.summary()
```

```
Model: "sequential_3"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━
┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━
┩
│ embedding_3 (Embedding)              │ (None, 50, 128)             │
1,280,000 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ gru_3 (GRU)                          │ (None, 64)                  │
37,248 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ dropout_2 (Dropout)                  │ (None, 64)                  │
0 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ dense_3 (Dense)                      │ (None, 1)                   │
```

```
65 │
└──────────────────────────────────┴──────────────────────────┴──────────
──────┘

 Total params: 1,317,313 (5.03 MB)

 Trainable params: 1,317,313 (5.03 MB)

 Non-trainable params: 0 (0.00 B)
```

```
# ========================================
# Train GRU
# ========================================

early_stop = EarlyStopping(
    monitor="val_loss",
    patience=2,
    restore_best_weights=True
)

history_gru_headline = model_gru_headline.fit(
    X_train_pad,
    y_train,
    validation_data=(X_val_pad, y_val),
    epochs=10,
    batch_size=128,
    callbacks=[early_stop],
    verbose=1
)
```

```
Epoch 1/10
336/336 ━━━━━━━━━━━━━━━━━━━━ 49s 138ms/step - accuracy: 0.8717 - loss:
0.2920 - val_accuracy: 0.9083 - val_loss: 0.2233
Epoch 2/10
336/336 ━━━━━━━━━━━━━━━━━━━━ 45s 133ms/step - accuracy: 0.9324 - loss:
0.1725 - val_accuracy: 0.9095 - val_loss: 0.2258
Epoch 3/10
336/336 ━━━━━━━━━━━━━━━━━━━━ 44s 130ms/step - accuracy: 0.9513 - loss:
0.1297 - val_accuracy: 0.9058 - val_loss: 0.2432
```

```
# ========================================
# GRU — HEADLINE ONLY
# Test Evaluation
# ========================================

y_test_prob_gru = model_gru_headline.predict(
    X_test_pad,
    verbose=1
).ravel()

y_test_pred_gru = (y_test_prob_gru >= 0.5).astype(int)
```

```python
print("\nGRU HEADLINE-ONLY TEST RESULTS")
print("------------------------------")

print("Accuracy :", accuracy_score(y_test, y_test_pred_gru))
print("Precision:", precision_score(y_test, y_test_pred_gru))
print("Recall   :", recall_score(y_test, y_test_pred_gru))
print("F1 Score :", f1_score(y_test, y_test_pred_gru))
print("ROC-AUC  :", roc_auc_score(y_test, y_test_prob_gru))
```

```
448/448 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step

GRU HEADLINE-ONLY TEST RESULTS
------------------------------
Accuracy : 0.905837119888151
Precision: 0.8929231581726961
Recall   : 0.9265652829154679
F1 Score : 0.9094332011026692
ROC-AUC  : 0.9683810838825758
```

```python
# ========================================
# GRU — CONTENT ONLY
# ========================================

model_gru_content = Sequential([
    Input(shape=(MAX_LEN,)),

    Embedding(
        input_dim=MAX_WORDS,
        output_dim=128,
        mask_zero=True
    ),

    GRU(64),

    Dropout(0.3),

    Dense(1, activation="sigmoid")
])

model_gru_content.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

model_gru_content.summary()
```

```
Model: "sequential_4"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━
┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━
┩
│ embedding_4 (Embedding)              │ (None, 50, 128)             │
1,280,000 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ gru_4 (GRU)                          │ (None, 64)                  │
37,248 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ dropout_3 (Dropout)                  │ (None, 64)                  │
0 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ dense_4 (Dense)                      │ (None, 1)                   │
65 │
└──────────────────────────────────────┴─────────────────────────────┴─────────────────
┘

 Total params: 1,317,313 (5.03 MB)

 Trainable params: 1,317,313 (5.03 MB)

 Non-trainable params: 0 (0.00 B)
```

```
# Show variables related to the content GRU data
[name for name in globals() if "content" in name.lower() or
name.endswith("_c")]
```

```
['content_df',
 'X_content',
 'y_content',
 'X_temp_c',
 'X_test_c',
 'y_temp_c',
 'y_test_c',
 'X_train_c',
 'X_val_c',
 'y_train_c',
 'y_val_c',
 'tfidf_content',
 'X_train_content_tfidf',
 'X_val_content_tfidf',
 'X_test_content_tfidf',
 'lr_content',
 'y_val_pred_lr_c',
```

```
 'y_test_pred_lr_c',
 'y_val_prob_lr_c',
 'y_test_prob_lr_c',
 'rf_content',
 'y_val_pred_rf_c',
 'y_test_pred_rf_c',
 'y_val_prob_rf_c',
 'y_test_prob_rf_c',
 'lgbm_content',
 'y_val_pred_lgbm_c',
 'y_test_pred_lgbm_c',
 'y_val_prob_lgbm_c',
 'y_test_prob_lgbm_c',
 'model_gru_content']
```

```
# ========================================
# CONTENT ONLY — TOKENIZATION
# ========================================

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

tokenizer_content = Tokenizer(
    num_words=MAX_WORDS,
    oov_token="<OOV>"
)

# IMPORTANT: fit only on training content
tokenizer_content.fit_on_texts(X_train_c)

# Convert text to integer sequences
X_train_content_seq = tokenizer_content.texts_to_sequences(X_train_c)
X_val_content_seq = tokenizer_content.texts_to_sequences(X_val_c)
X_test_content_seq = tokenizer_content.texts_to_sequences(X_test_c)

print("Content vocabulary size:", len(tokenizer_content.word_index))

print("\nExample content sequence:")
print(X_train_content_seq[0])
```

```
Content vocabulary size: 294855

Example content sequence:
[1, 62, 6, 520, 89, 195, 1471, 4655, 4592, 9, 32, 87, 819, 561, 8374,
1916, 3948, 88, 479, 508, 7643, 3175, 1762, 819, 125, 402, 6302, 37,
561, 3175, 557, 5407, 25, 300, 1328, 3624, 3427, 217, 1579, 37, 561,
359, 1005, 2065, 946, 89, 3, 912, 374, 137, 676, 1049, 833, 97, 222,
520, 89, 1666, 561, 21, 4772, 223, 3782, 1, 1, 9600, 19, 2628, 313,
7149, 1255, 666, 176, 479, 508, 4096, 5, 383, 2063, 31, 4, 823, 254,
5805, 3400, 2, 780, 1, 169, 646, 1, 1063, 5, 1625, 1397, 574, 1968,
```

```
31, 4, 169, 922, 1434, 2, 2200, 50, 1425, 219, 345, 6, 1584, 373,
3186, 60, 2509, 4286, 3068, 103, 174, 50, 585, 4159, 2872, 638, 3637,
19, 48, 930, 1, 41, 62, 6, 6197, 5947, 3573, 1, 1104, 7, 1719, 353,
1397, 2397, 62, 267, 669, 7, 479, 508, 2548, 244, 1493, 5214, 31, 4,
407, 666, 550, 199, 41, 62, 191, 199, 2, 6, 1269, 5, 372, 3244, 1897,
4065, 576, 270, 2548, 501, 506, 672, 1, 5122, 2548, 1394, 2548, 2821,
1, 1228, 60, 2531, 43, 1397, 6, 1584, 1394, 646, 3016, 70, 1584, 300,
15, 1066, 400, 2178, 312, 388, 48, 110, 1, 2, 1, 780, 3021, 6039, 305,
1397, 2143, 1, 190, 272, 2736, 517, 821, 1, 2243, 105, 1228, 60, 780,
1, 15, 2, 2262, 43, 1397, 2580, 2620, 1506, 31, 4, 149, 391, 43, 1397,
685, 3683, 41, 62, 336, 174, 50, 397, 49, 339, 1580, 5, 7402, 300, 25,
1]

# ========================================
# CONTENT ONLY — PADDING
# ========================================

X_train_content_pad = pad_sequences(
    X_train_content_seq,
    maxlen=MAX_LEN,
    padding="post",
    truncating="post"
)

X_val_content_pad = pad_sequences(
    X_val_content_seq,
    maxlen=MAX_LEN,
    padding="post",
    truncating="post"
)

X_test_content_pad = pad_sequences(
    X_test_content_seq,
    maxlen=MAX_LEN,
    padding="post",
    truncating="post"
)

print("Training shape:", X_train_content_pad.shape)
print("Validation shape:", X_val_content_pad.shape)
print("Test shape:", X_test_content_pad.shape)

Training shape: (42423, 50)
Validation shape: (14141, 50)
Test shape: (14141, 50)

from tensorflow.keras.callbacks import EarlyStopping

# Define the missing early stopping callback for the content model
early_stop_content = EarlyStopping(
```

```
    monitor="val_loss",
    patience=2,
    restore_best_weights=True
)

# Train the GRU model on Content-Only data
history_gru_content = model_gru_content.fit(
    X_train_content_pad,
    y_train_c,
    validation_data=(X_val_content_pad, y_val_c),
    epochs=10,
    batch_size=128,
    callbacks=[early_stop_content],
    verbose=1
)
```

```
Epoch 1/10
332/332 ━━━━━━━━━━━━━━━━━━━━ 48s 134ms/step - accuracy: 0.8784 - loss:
0.2718 - val_accuracy: 0.9359 - val_loss: 0.1709
Epoch 2/10
332/332 ━━━━━━━━━━━━━━━━━━━━ 44s 132ms/step - accuracy: 0.9555 - loss:
0.1264 - val_accuracy: 0.9324 - val_loss: 0.1755
Epoch 3/10
332/332 ━━━━━━━━━━━━━━━━━━━━ 44s 132ms/step - accuracy: 0.9704 - loss:
0.0874 - val_accuracy: 0.9345 - val_loss: 0.2106
```

```
# ========================================
# GRU — CONTENT ONLY
# Test Evaluation
# ========================================

y_test_prob_gru_c = model_gru_content.predict(
    X_test_content_pad,
    verbose=1
).ravel()

y_test_pred_gru_c = (y_test_prob_gru_c >= 0.5).astype(int)

print("\nGRU CONTENT-ONLY TEST RESULTS")
print("------------------------------")

print("Accuracy :", accuracy_score(y_test_c, y_test_pred_gru_c))
print("Precision:", precision_score(y_test_c, y_test_pred_gru_c))
print("Recall   :", recall_score(y_test_c, y_test_pred_gru_c))
print("F1 Score :", f1_score(y_test_c, y_test_pred_gru_c))
print("ROC-AUC  :", roc_auc_score(y_test_c, y_test_prob_gru_c))
```

```
442/442 ━━━━━━━━━━━━━━━━━━━━ 6s 12ms/step

GRU CONTENT-ONLY TEST RESULTS
------------------------------
```

```
Accuracy : 0.9352238172689343
Precision: 0.9206113064647011
Recall   : 0.953895739910314
F1 Score : 0.9369580178940123
ROC-AUC  : 0.9814316647621975
```

```
# ========================================
# GRU — HEADLINE + CONTENT
# Tokenization
# ========================================

tokenizer_combined = Tokenizer(
    num_words=MAX_WORDS,
    oov_token="<OOV>"
)

# Fit ONLY on combined training data
tokenizer_combined.fit_on_texts(X_train_comb)

# Convert text to sequences
X_train_comb_seq = tokenizer_combined.texts_to_sequences(X_train_comb)
X_val_comb_seq = tokenizer_combined.texts_to_sequences(X_val_comb)
X_test_comb_seq = tokenizer_combined.texts_to_sequences(X_test_comb)

print("Combined vocabulary size:", len(tokenizer_combined.word_index))

print("\nExample combined sequence:")
print(X_train_comb_seq[0])
```

```
Combined vocabulary size: 301908

Example combined sequence:
[455, 594, 6439, 1821, 16, 2000, 4176, 4247, 507, 3, 74, 455, 594, 1,
1, 58, 905, 5814, 136, 29, 3, 4247, 219, 1, 3364, 810, 1090, 909, 29,
3, 16, 43, 2637, 2103, 909, 387, 909, 295, 2103, 481, 506, 838, 597,
909, 4247, 60, 1470, 1, 189, 1821, 424, 2329, 4176, 540, 365, 4052,
1430, 1, 2, 853, 8524, 154, 1821, 16, 158, 662, 178, 4247, 631, 4921,
49, 1, 412, 43, 661, 437, 7289, 3930, 3441, 1102, 43, 4911, 104, 6376,
6, 281, 642, 3365, 1, 4290, 6923, 1412, 4082, 3591, 258, 738, 3926,
606, 60, 5607, 1, 2250, 2489, 1, 5508, 3, 1169, 352, 160, 1, 3575, 3,
2123, 30, 224, 99, 275, 8347, 542, 387, 178, 542, 1, 19, 876, 98,
4569, 1430, 1850, 1917, 939, 4951, 3419, 431, 1478, 7063, 1780, 593,
16, 5060, 81, 290, 1292, 4717, 6798, 6639, 166, 3, 480, 1, 1771, 16,
2946, 9, 24, 3781, 3, 909, 412, 410, 1075, 299, 2692, 16, 1361, 27,
365, 2160, 7487, 4600, 2894, 119, 1, 1, 6583, 1, 3870, 1593, 3, 2731,
2329, 1, 3017, 139, 9707, 1, 12, 604, 1, 226, 16, 90, 2454, 4176, 3,
36, 12, 2657, 365, 1850, 507, 909, 431, 1, 36, 717, 74, 222, 1, 290,
1821, 16, 3598, 1676, 160, 133, 4247, 412, 43, 149, 159, 4247, 3534,
1, 141, 222, 74, 1464, 1729]
```

```
# ========================================
# HEADLINE + CONTENT — PADDING
# ========================================

X_train_comb_pad = pad_sequences(
    X_train_comb_seq,
    maxlen=MAX_LEN,
    padding="post",
    truncating="post"
)

X_val_comb_pad = pad_sequences(
    X_val_comb_seq,
    maxlen=MAX_LEN,
    padding="post",
    truncating="post"
)

X_test_comb_pad = pad_sequences(
    X_test_comb_seq,
    maxlen=MAX_LEN,
    padding="post",
    truncating="post"
)

print("Training shape:", X_train_comb_pad.shape)
print("Validation shape:", X_val_comb_pad.shape)
print("Test shape:", X_test_comb_pad.shape)
```

```
Training shape: (42921, 50)
Validation shape: (14308, 50)
Test shape: (14308, 50)
```

```
# ========================================
# GRU — HEADLINE + CONTENT
# Rebuild with cuDNN disabled
# ========================================

model_gru_combined = Sequential([
    Input(shape=(MAX_LEN,)),

    Embedding(
        input_dim=MAX_WORDS,
        output_dim=128,
        mask_zero=True
    ),

    GRU(
        64,
        use_cudnn=False
```

```
    ),

    Dropout(0.3),

    Dense(1, activation="sigmoid")
])

model_gru_combined.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

model_gru_combined.summary()
```

```
Model: "sequential_5"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━┓
┃ Layer (type)                         ┃ Output Shape                ┃         Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━┩
│ embedding_5 (Embedding)              │ (None, 50, 128)             │       1,280,000 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ gru_5 (GRU)                          │ (None, 64)                  │          37,248 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ dropout_4 (Dropout)                  │ (None, 64)                  │               0 │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ dense_5 (Dense)                      │ (None, 1)                   │              65 │
└──────────────────────────────────────┴─────────────────────────────┴─────────────────┘
```

```
 Total params: 1,317,313 (5.03 MB)

 Trainable params: 1,317,313 (5.03 MB)

 Non-trainable params: 0 (0.00 B)
```

```
# ========================================
# Train GRU — Headline + Content
# ========================================

early_stop_combined = EarlyStopping(
```

```
    monitor="val_loss",
    patience=2,
    restore_best_weights=True
)

history_gru_combined = model_gru_combined.fit(
    X_train_comb_pad,
    y_train_comb,
    validation_data=(X_val_comb_pad, y_val_comb),
    epochs=10,
    batch_size=128,
    callbacks=[early_stop_combined],
    verbose=1
)
```

```
Epoch 1/10
336/336 ━━━━━━━━━━━━━━━━━━━━ 47s 132ms/step - accuracy: 0.9026 - loss:
0.2306 - val_accuracy: 0.9547 - val_loss: 0.1334
Epoch 2/10
336/336 ━━━━━━━━━━━━━━━━━━━━ 46s 137ms/step - accuracy: 0.9686 - loss:
0.0960 - val_accuracy: 0.9567 - val_loss: 0.1249
Epoch 3/10
336/336 ━━━━━━━━━━━━━━━━━━━━ 44s 131ms/step - accuracy: 0.9793 - loss:
0.0675 - val_accuracy: 0.9560 - val_loss: 0.1318
Epoch 4/10
336/336 ━━━━━━━━━━━━━━━━━━━━ 44s 130ms/step - accuracy: 0.9848 - loss:
0.0498 - val_accuracy: 0.9577 - val_loss: 0.1398
```

```
# ============================================
# Evaluate GRU - Headline + Content
# ============================================

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report
)

# Predict probabilities on the TEST dataset
y_prob_gru = model_gru_combined.predict(
    X_test_comb_pad,
    verbose=0
).ravel()

# Convert probabilities to class predictions
y_pred_gru = (y_prob_gru >= 0.5).astype(int)
```

```python
# Calculate evaluation metrics
accuracy_gru = accuracy_score(y_test_comb, y_pred_gru)
precision_gru = precision_score(y_test_comb, y_pred_gru,
zero_division=0)
recall_gru = recall_score(y_test_comb, y_pred_gru, zero_division=0)
f1_gru = f1_score(y_test_comb, y_pred_gru, zero_division=0)
auc_gru = roc_auc_score(y_test_comb, y_prob_gru)

# Display results
print("GRU - Headline + Content Test Results")
print("--------------------------------------")
print(f"Accuracy  : {accuracy_gru:.4f}")
print(f"Precision : {precision_gru:.4f}")
print(f"Recall    : {recall_gru:.4f}")
print(f"F1-score  : {f1_gru:.4f}")
print(f"ROC-AUC   : {auc_gru:.4f}")
```

```
GRU - Headline + Content Test Results
--------------------------------------
Accuracy  : 0.9526
Precision : 0.9522
Recall    : 0.9551
F1-score  : 0.9536
ROC-AUC   : 0.9882
```

```python
# GRU - Macro-averaged test metrics

from sklearn.metrics import precision_score, recall_score, f1_score

precision_gru_macro = precision_score(
    y_test_comb, y_pred_gru, average="macro", zero_division=0
)

recall_gru_macro = recall_score(
    y_test_comb, y_pred_gru, average="macro", zero_division=0
)

f1_gru_macro = f1_score(
    y_test_comb, y_pred_gru, average="macro", zero_division=0
)

print("GRU - Headline + Content (Macro Average)")
print(f"Accuracy  : {accuracy_gru:.4f}")
print(f"Precision : {precision_gru_macro:.4f}")
print(f"Recall    : {recall_gru_macro:.4f}")
print(f"F1-score  : {f1_gru_macro:.4f}")
print(f"ROC-AUC   : {auc_gru:.4f}")
```

```
GRU - Headline + Content (Macro Average)
Accuracy  : 0.9526
```

```
Precision : 0.9526
Recall    : 0.9526
F1-score  : 0.9526
ROC-AUC   : 0.9882
```

```
import pandas as pd
import numpy as np

print("=== DATAFRAMES CONTAINING NEWS TEXT ===")

dataframes_found = []

for name, obj in list(globals().items()):
    if isinstance(obj, pd.DataFrame):
        cols = [str(c).lower() for c in obj.columns]

        has_title = any("title" in c or "headline" in c for c in cols)
        has_content = any(
            "content" in c or "text" in c or "article" in c
            for c in cols
        )

        if has_title and has_content:
            dataframes_found.append((name, obj))
            print(f"\nVariable: {name}")
            print("Shape:", obj.shape)
            print("Columns:", obj.columns.tolist())

            label_cols = [
                c for c in obj.columns
                if str(c).lower() in ["label", "labels", "target",
"class"]
            ]

            if label_cols:
                print("Label column:", label_cols[0])
                print("Label counts:")
                print(obj[label_cols[0]].value_counts(dropna=False))

print("\n=== EXISTING TRAIN / VALIDATION / TEST VARIABLES ===")

for name in [
    "X_train_comb", "X_val_comb", "X_test_comb",
    "X_train_comb_pad", "X_val_comb_pad", "X_test_comb_pad",
    "y_train_comb", "y_val_comb", "y_test_comb"
]:
    if name in globals():
        obj = globals()[name]

        print(f"\n{name}")
```

```
        print("Type:", type(obj).__name__)

        if hasattr(obj, "shape"):
            print("Shape:", obj.shape)

        try:
            print("First value:", obj.iloc[0] if isinstance(obj,
pd.Series)
                  else obj[0])
        except (IndexError, KeyError, TypeError):
            print("First value: unavailable")

=== DATAFRAMES CONTAINING NEWS TEXT ===

Variable: ___
Shape: (5, 3)
Columns: ['title', 'text', 'label']
Label column: label
Label counts:
label
1    4
0    1
Name: count, dtype: int64

Variable: df
Shape: (71537, 5)
Columns: ['title', 'text', 'label', 'clean_title', 'clean_text']
Label column: label
Label counts:
label
1    36509
0    35028
Name: count, dtype: int64

Variable: _4
Shape: (5, 3)
Columns: ['title', 'text', 'label']
Label column: label
Label counts:
label
1    4
0    1
Name: count, dtype: int64

=== EXISTING TRAIN / VALIDATION / TEST VARIABLES ===

X_train_comb
Type: Series
Shape: (42921,)
First value: cnn host destroys moderate republican failing condemn
```

extremism led trump video cnn host fareed zakaria made pretty damning point donald trump extremism monday nightzakaria dedicated minute discus rise donald trump republican party pointing extremist rise different rise islamic extremist middle east main cause rise extremism world islam cowardice muslim moderate decade chose condemn bad idea ugly rhetoric zakaria said clearly referencing fact moderate republican home allowed conservative extremism christian sharia law fester within party turned full blown infection threatens kill party ruin america fearing would seen simply ideological weakling avoided confronting cancer plain sight clear similar dynamic play world conservatism continuedzakaria praised mitt romeny condemning trump criticized taking long gratefully accepting trump endorsement even though still face birther movement different conservative movement zakaria say mainstream member embraced rhetoric tactic extreme noted newt gingrich began shift circulating memo calling republican smear democrat word failure pathetic disgrace incompetent used trump current electionzaharia destroyed republican notion president obama fault trump rise within gop offered better explanation republican fed country idea decline betrayal treason encouraged force antiintellectualism obstructionism populism flirted bigotry racism trump merely chose unashamedly embrace saying plainly hinting year hit jackpot problem republican leader begun condemn trump last year condemned idea tactic led rise began flourish last century video via youtubein word moderate republican apologize waiting long call extremism within party question whether extremism defeated latefeatured image via video screen capture

X_val_comb
Type: Series
Shape: (14308,)
First value: joe scarborough reference ‘failed’ obama presidency amazingly racist term video morning joe msnbc supposedly leftwing political news opinion network host joe scarborough style moderate conservative willing criticize nutjob faction party many instance dress leftleaning cohosts like mika brzezinski willie geist along frequent guest also democrat like former democratic congressman harold ford jr however scarborough never let u forget still republican still hold least level gop despicable bigoted view definitely display wednesday morning talk turned president obama legacy overall presidency comparison supposedly improving approval number newly elected orange buffoon donald trump scarborough made shockingly racist statement obama came black jesus want come redneck jesus president united state want come people like ronald reagan actually underestimate medium done donald trump great favor told u past week white supremacist nazi bringing nazi back america scarborough ended tirade calling colleague press fool attempt derail bludgeon trump wrong way set bar low donald trump say bar going walk one thing joe scarborough racist get course high expectation president obama first african american man elected twice record number matter one think man

policy prescription fact alone make presidency historic one put amazing amount pressure upon himsecondly presidency far failure despite fact gop senate leader mitch mcconnell rest obstructionist party one goal one goal mind make sure obama failed lasted one term still got plenty done cleaned economic mess george w bush left nation world got record number american health insurance made military safer place lgbtq people got rid bigoted defense marriage act secured marriage equality much happened president obama watch party obstructionist blocking wayas donald trump one called trump nazi however denying spent entire ugly repugnant campaign playing footsie neonazis klan member assorted racist bigot hell done good job actually gall put literal white supremacist step oval office nominate man deemed racist federal judge run entire justice department denying direction trump swing come raceso word check racism joe scarborough dead wrong one one piece proof even socalled reasonable republican still racist comewatch full exchange herefeatured image via video screen capture

X_test_comb
Type: Series
Shape: (14308,)
First value: huckabee nail say obama’s radical list invited guest meet pope prof fake christianity welcoming prolife promarriage leader white house crowd abortion gay right activist classy hosting alcoholic anonymous meeting open bar mike huckabee said tuesday barack obama pretend christian president guest list pope francis white house welcome ceremony included short list catholic christian haev visibly parted church doctrine huckabee told newsmax tv concerned guy belief christian pretend say thing make difficult people practice christian faith disappointed person say christian invite pope home invite whole bunch people odds catholic church policy think something unseemly guest list wednesday morning pomp circumstance south lawn white house included prochoice nun transgender woman openly gay retired episcopal bishop plus roman catholic teacher suburban philadelphia fired year marrying girlfriendgene robinson first openly gay episcopal bishop divorced partner last year previously divorcing first spouse womanmargie winter suburban philadelphia educator said monday hoped pontiff would address issue facing gay catholic visit usthe yearold winter lost job religious instruction director waldron mercy academy montgomery county pennsylvania june samesex marriagewinters say school principal knew gay married andrea vettori clinical director health center homeless hired eight year agoher invitation join crowd wednesday morning came via human right campaign gayrights activist grouphuckabee answering question tuesday night whether muslim politician disqualified running president religious test person hold public office america said less concerned faith person concerned authenticity faith play policy huckabee lashed obama tuesday essay published daily caller saying show total disrespect million american transforming pope francis white house visit politicized cattle call

```
gay proabortion activist via uk daily mail online

X_train_comb_pad
Type: ndarray
Shape: (42921, 50)
First value: [ 455  594 6439 1821   16 2000 4176 4247  507    3   74
455  594    1
    1   58  905 5814  136   29    3 4247  219    1 3364  810 1090  909
   29    3   16   43 2637 2103  909  387  909  295 2103  481  506  838
  597  909 4247   60 1470    1  189 1821]

X_val_comb_pad
Type: ndarray
Shape: (14308, 50)
First value: [1192 5383 1831    1   24  660    1  674  374   74  533
1192 2054 3306
 4056   53   40  761  554  594 1192 5383 2177 1821  178 1216 4214    1
 3352   43   34 1709 3130 8302    1   17 7842 6947    1    1  373 3776
 1795   15   81   17   55  111 1876 9308]

X_test_comb_pad
Type: ndarray
Shape: (14308, 50)
First value: [4281 5344   19  670 1202  502 2336 1795  680 2061 3388
831 4356 5849
 7667    1   90   33   25  795 1030 1051   38  715    1 5307    1 2762
  208  353 1329  940 4281    2  181  414   24 4245  631    9 1795  502
 2061 2786   33   25 2021 2393  821  812]

y_train_comb
Type: Series
Shape: (42921,)
First value: 1

y_val_comb
Type: Series
Shape: (14308,)
First value: 1

y_test_comb
Type: Series
Shape: (14308,)
First value: 1

# Check whether the original combined text is still available

for name in [
    "X_train_comb", "X_val_comb", "X_test_comb"
]:
    print(f"\n--- {name} ---")
```

```
    if name in globals():
        data = globals()[name]
        print("Type:", type(data).__name__)
        print("Shape:", getattr(data, "shape", "Not available"))

        if len(data) > 0:
            print("First sample:", repr(data[0] if isinstance(
                data, (list, np.ndarray)
            ) else data.iloc[0])[:500])
    else:
        print("Variable not found")


--- X_train_comb ---
Type: Series
Shape: (42921,)
First sample: 'cnn host destroys moderate republican failing condemn
extremism led trump video cnn host fareed zakaria made pretty damning
point donald trump extremism monday nightzakaria dedicated minute
discus rise donald trump republican party pointing extremist rise
different rise islamic extremist middle east main cause rise extremism
world islam cowardice muslim moderate decade chose condemn bad idea
ugly rhetoric zakaria said clearly referencing fact moderate
republican home allowed conservative extrem

--- X_val_comb ---
Type: Series
Shape: (14308,)
First sample: 'joe scarborough reference ‘failed’ obama presidency
amazingly racist term video morning joe msnbc supposedly leftwing
political news opinion network host joe scarborough style moderate
conservative willing criticize nutjob faction party many instance
dress leftleaning cohosts like mika brzezinski willie geist along
frequent guest also democrat like former democratic congressman harold
ford jr however scarborough never let u forget still republican still
hold least level gop despicable bigoted v

--- X_test_comb ---
Type: Series
Shape: (14308,)
First sample: 'huckabee nail say obama’s radical list invited guest
meet pope prof fake christianity welcoming prolife promarriage leader
white house crowd abortion gay right activist classy hosting alcoholic
anonymous meeting open bar mike huckabee said tuesday barack obama
pretend christian president guest list pope francis white house
welcome ceremony included short list catholic christian haev visibly
parted church doctrine huckabee told newsmax tv concerned guy belief
christian pretend say thing make dif
```

```python
import torch
import transformers
import numpy as np

from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification
)

print("PyTorch version:", torch.__version__)
print("Transformers version:", transformers.__version__)
print("GPU available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU name:", torch.cuda.get_device_name(0))

print("\n=== LABEL CHECK ===")

for name, labels in [
    ("Training", y_train_comb),
    ("Validation", y_val_comb),
    ("Test", y_test_comb)
]:
    print(f"\n{name} labels:")
    print(labels.value_counts().sort_index())

    print("Unique values:", sorted(labels.unique().tolist()))

print("\n=== TEXT CHECK ===")
print("Training samples:", len(X_train_comb))
print("Validation samples:", len(X_val_comb))
print("Test samples:", len(X_test_comb))
print("Example combined text:")
print(X_train_comb.iloc[0][:300])
```

```
PyTorch version: 2.11.0+cpu
Transformers version: 5.18.0
GPU available: False

=== LABEL CHECK ===

Training labels:
label
0    21016
1    21905
Name: count, dtype: int64
Unique values: [0, 1]

Validation labels:
label
```

```
0    7006
1    7302
Name: count, dtype: int64
Unique values: [0, 1]

Test labels:
label
0    7006
1    7302
Name: count, dtype: int64
Unique values: [0, 1]

=== TEXT CHECK ===
Training samples: 42921
Validation samples: 14308
Test samples: 14308
Example combined text:
cnn host destroys moderate republican failing condemn extremism led
trump video cnn host fareed zakaria made pretty damning point donald
trump extremism monday nightzakaria dedicated minute discus rise
donald trump republican party pointing extremist rise different rise
islamic extremist middle east


# Find the original dataframe containing news text and labels

for name, df in list(globals().items()):
    if not isinstance(df, pd.DataFrame):
        continue

    columns = {str(c).strip().lower(): c for c in df.columns}

    title_col = next(
        (columns[c] for c in columns
         if "title" in c or "headline" in c),
        None
    )

    content_col = next(
        (columns[c] for c in columns
         if "content" in c or "text" in c or "article" in c),
        None
    )

    label_col = next(
        (columns[c] for c in columns
         if c in ["label", "labels", "target", "class"]),
        None
    )
```

```
    if title_col and content_col and label_col:
        print(f"\nDataframe found: {name}")
        print("Shape:", df.shape)
        print("Columns:", df.columns.tolist())
        print("Dataframe index sample:", df.index[:5].tolist())

        for label in sorted(df[label_col].dropna().unique()):
            row = df[df[label_col] == label].iloc[0]
            print(f"\nLabel {label} example:")
            print("Title:", str(row[title_col])[:150])
            print("Content:", str(row[content_col])[:200])


Dataframe found: ___
Shape: (5, 3)
Columns: ['title', 'text', 'label']
Dataframe index sample: [0, 1, 2, 3, 4]

Label 0 example:
Title: Bobby Jindal, raised Hindu, uses story of Christian conversion
to woo evangelicals for potential 2016 bid
Content: A dozen politically active pastors came here for a private
dinner Friday night to hear a conversion story unique in the context
of presidential politics: how Louisiana Gov. Bobby Jindal traveled
from

Label 1 example:
Title: LAW ENFORCEMENT ON HIGH ALERT Following Threats Against Cops
And Whites On 9-11By #BlackLivesMatter And #FYF911 Terrorists [VIDEO]
Content: No comment is expected from Barack Obama Members of the
#FYF911 or #FukYoFlag and #BlackLivesMatter movements called for the
lynching and hanging of white people and cops. They encouraged others
on a

Dataframe found: df
Shape: (71537, 5)
Columns: ['title', 'text', 'label', 'clean_title', 'clean_text']
Dataframe index sample: [0, 1, 2, 3, 4]

Label 0 example:
Title: Bobby Jindal, raised Hindu, uses story of Christian conversion
to woo evangelicals for potential 2016 bid
Content: A dozen politically active pastors came here for a private
dinner Friday night to hear a conversion story unique in the context
of presidential politics: how Louisiana Gov. Bobby Jindal traveled
from

Label 1 example:
Title: LAW ENFORCEMENT ON HIGH ALERT Following Threats Against Cops
And Whites On 9-11By #BlackLivesMatter And #FYF911 Terrorists [VIDEO]
```

```
Content: No comment is expected from Barack Obama Members of the
#FYF911 or #FukYoFlag and #BlackLivesMatter movements called for the
lynching and hanging of white people and cops. They encouraged others
on a

Dataframe found: _4
Shape: (5, 3)
Columns: ['title', 'text', 'label']
Dataframe index sample: [0, 1, 2, 3, 4]

Label 0 example:
Title: Bobby Jindal, raised Hindu, uses story of Christian conversion
to woo evangelicals for potential 2016 bid
Content: A dozen politically active pastors came here for a private
dinner Friday night to hear a conversion story unique in the context
of presidential politics: how Louisiana Gov. Bobby Jindal traveled
from

Label 1 example:
Title: LAW ENFORCEMENT ON HIGH ALERT Following Threats Against Cops
And Whites On 9-11By #BlackLivesMatter And #FYF911 Terrorists [VIDEO]
Content: No comment is expected from Barack Obama Members of the
#FYF911 or #FukYoFlag and #BlackLivesMatter movements called for the
lynching and hanging of white people and cops. They encouraged others
on a


# Recover the original dataframe without overwriting it again

df_candidates = [
    (name, obj)
    for name, obj in dataframes_found
    if isinstance(obj, pd.DataFrame)
    and {"title", "text", "label", "clean_title", "clean_text"}
        .issubset(set(obj.columns))
]

print("Matching dataframes:")
for name, obj in df_candidates:
    print(name, obj.shape)

if not df_candidates:
    raise ValueError(
        "Original dataframe not found in dataframes_found. "
        "Send me the output and we will recover it another way."
    )

# Select the full dataset
original_df = max(df_candidates, key=lambda item: len(item[1]))[1]
```

```python
print("\nRecovered dataset shape:", original_df.shape)
print("Columns:", original_df.columns.tolist())

# Verify that the existing splits point to rows in this dataset
print("\n=== SPLIT ALIGNMENT ===")

for name, X in [
    ("Train", X_train_comb),
    ("Validation", X_val_comb),
    ("Test", X_test_comb)
]:
    indices_exist = X.index.isin(original_df.index).all()

    print(f"{name}:")
    print("  Samples:", len(X))
    print("  All indices exist in original dataframe:", indices_exist)
    print("  First indices:", X.index[:5].tolist())
```

```
Matching dataframes:
df (71537, 5)

Recovered dataset shape: (71537, 5)
Columns: ['title', 'text', 'label', 'clean_title', 'clean_text']

=== SPLIT ALIGNMENT ===
Train:
  Samples: 42921
  All indices exist in original dataframe: True
  First indices: [51214, 1121, 28248, 54533, 8331]
Validation:
  Samples: 14308
  All indices exist in original dataframe: True
  First indices: [39908, 50706, 58340, 16434, 32280]
Test:
  Samples: 14308
  All indices exist in original dataframe: True
  First indices: [10880, 24161, 12049, 1270, 44661]
```

```python
print("\n=== ALIGNMENT CHECK ===")

# Confirm that the split indices exist in the original dataframe
for name, X in [
    ("Train", X_train_comb),
    ("Validation", X_val_comb),
    ("Test", X_test_comb)
]:
    indices_exist = X.index.isin(original_df.index).all()
    indices_unique = X.index.is_unique

    print(f"{name}:")
```

```
    print("  All indices exist in original_df:", indices_exist)
    print("  Indices are unique:", indices_unique)

# Check whether the existing GRU text was made from cleaned columns
clean_combined = (
    original_df["clean_title"].fillna("").astype(str).str.strip()
    + " "
    + original_df["clean_text"].fillna("").astype(str).str.strip()
).str.strip()

for name, X in [
    ("Train", X_train_comb),
    ("Validation", X_val_comb),
    ("Test", X_test_comb)
]:
    expected = clean_combined.loc[X.index]
    matches = (
        expected.reset_index(drop=True)
        == X.reset_index(drop=True)
    ).all()

    print(f"{name} cleaned text matches dataframe:", matches)


=== ALIGNMENT CHECK ===
Train:
  All indices exist in original_df: True
  Indices are unique: True
Validation:
  All indices exist in original_df: True
  Indices are unique: True
Test:
  All indices exist in original_df: True
  Indices are unique: True
Train cleaned text matches dataframe: False
Validation cleaned text matches dataframe: False
Test cleaned text matches dataframe: False


# Verify that labels match and the three splits do not overlap

splits = [
    ("Train", X_train_comb, y_train_comb),
    ("Validation", X_val_comb, y_val_comb),
    ("Test", X_test_comb, y_test_comb)
]

split_indices = {}

print("=== LABEL ALIGNMENT ===")
```

```python
for name, X, y in splits:
    # Get the labels from the original dataframe in the same row order
    expected = original_df.loc[X.index,
"label"].reset_index(drop=True)
    actual = y.reindex(X.index).reset_index(drop=True)

    # Compare labels after converting to integers
    expected = pd.to_numeric(expected).astype(int)
    actual = pd.to_numeric(actual).astype(int)

    print(f"{name}: labels match original dataframe =",
          expected.equals(actual))

    split_indices[name] = set(X.index)

print("\n=== SPLIT OVERLAP ===")

print("Train/Validation overlap:",
      len(split_indices["Train"] & split_indices["Validation"]))

print("Train/Test overlap:",
      len(split_indices["Train"] & split_indices["Test"]))

print("Validation/Test overlap:",
      len(split_indices["Validation"] & split_indices["Test"]))

all_indices = (
    split_indices["Train"]
    | split_indices["Validation"]
    | split_indices["Test"]
)

print("\nTotal unique samples across splits:", len(all_indices))
print("Original dataframe rows:", len(original_df))
print("All rows accounted for:", len(all_indices) == len(original_df))
```

```
=== LABEL ALIGNMENT ===
Train: labels match original dataframe = True
Validation: labels match original dataframe = True
Test: labels match original dataframe = True

=== SPLIT OVERLAP ===
Train/Validation overlap: 0
Train/Test overlap: 0
Validation/Test overlap: 0

Total unique samples across splits: 71537
Original dataframe rows: 71537
All rows accounted for: True
```